Authored by: Arpit Raj, LNMIIT Jaipur
Date: Wednesday, October 28, 2026
© 2026 Arpit Raj. All rights reserved.

C++ Object-Oriented Design

Abstraction & Virtual Behaviors

Abstraction vs Encapsulation

- **Abstraction** focuses on what an object offers and hides unnecessary implementation details achieved through well designed interfaces, classes and abstract classes.
  - Eg calling withdraw without knowing internal details (i.e. work)
- **Encapsulation** focuses on grouping data and methods and controlling access, protects internal state through access control.
  - Eg prevent access to balance
- **getters** are used when callers legitimately needs to read a value
- **setters** are used when caller should be able to replace a value

Private methods can help abstraction

```
private:
    bool verifyPin(string pin) { return pin == "Aadya"; }
public:
    void withdraw(string pin, int amount) {
        if (verifyPin(pin) && amount > 0) {
            cout << "done" << "\n";
        }
        else {
            cout << "chala ja BSDK" ;
        }
    }
```

hence caller can use `withdraw` but not `verifyPin`, which adds an additional secure layer and help abstraction.

Interfaces & Abstract Classes

Abstract classes and pure virtual functions (already covered)

- # an abstract class can have constructors, data members, other methods, and other non pure virtual functions as long as it have atleast one pure virtual function.

Make interfaces using Abstract classes
Eg

```
class Printable {
public:
    virtual void print() = 0;
    virtual ~Printable() = default;
};

class Doc : public Printable {
public:
    void print() override { cout << "Print Doc" << "\n"; }
};

class Photo : public Printable {
public:
    void print() override { cout << "Print Photo"; }
};
```

1. **defining contract:** `virtual void print() = 0;` means every derived class must provide their own implementation.
2. **Implement contract:** `void print() override` is the implementation provided by derived.

- the use of virtual destructor here is already justified in previous notes.

© Arpit Raj, LNMIIT Jaipur

Design Principles of Abstraction

3 DESIGN PRINCIPLES OF ABSTRACTION

1. **Hide implementation details**
   - know what operation does, not its internal steps
2. **Protect valid state**
   - dont expose internal data, if exposing them can lead to violation of class rules
3. **Keep public API focussed**
   - avoid unnecessary getters, setters, methods, provide operations actually needed

Important Notes & Edge Cases

- # **Note:** A derived class can remain abstract:

Eg

```
class Animal { public: virtual void sound() = 0; };
class Dog : public Animal { };
```

here Dog is still abstract because it hasnt implemented sound, so `Dog d` will give compilation error.

Note

```
class Base { public: virtual ~Base() = 0; };
class child : public Base { public: ~child() { /* code */ } };
```

now in main if we write :-

```
Base * ptr = new Derived();
delete ptr;
```

⚠️ IMPORTANT - NOTES (PURE VIRTUAL DESTRUCTOR)

this will give an error, `undefined reference to Base::~Base()`
because compiler lets declare class, when child deleted, it destroys derived part and then calls base class destructor, and here base class has no actual body for its destructor.

solution :

```
Base::~Base() {
}
```

write this anywhere outside class definitions but not in main function.

🛑 O/P Q: AVOID CALLING VIRTUAL FUNCTIONS INSIDE A CONSTRUCTOR OR DESTRUCTOR

Eg

```
class Animal { public:
    Animal() { sound(); }
    virtual ~Animal() { sound(); }
    virtual void sound() { cout << "Animal sound "; }
};

class Dog: public Animal { public:
    Dog() { }
    void sound() override { std::cout << "Dog sound"; }
};
```

now if `Dog MyDog;`
then
Base class constructor called first, at that time no Dog. So **O/p = Animal sound**
and then derived destructor called first Dog is deleted and then Animal destructor but Dog deleted / only animal remains so again **"Animal sound"**.

© Arpit Raj, LNMIIT Jaipur